

Introduction:

Symbol	Hexa code	Description
AND	0 or 8	AND M to AC
ADD	1 or 9	Add M to AC, carry to E
LDA	2 or A	Load AC from M
STA	3 or B	Store AC in M
BUN	4 or C	Branch unconditionally to m
BSA	5 or D	Save return address in m and branch to m+1
ISZ	6 or E	Increment M and skip if zero
CLA	7800	Clear AC
CLE	7400	Clear E
CMA	7200	Complement AC
CME	7100	Complement E
CIR	7080	Circulate right E and AC
CIL	7040	Circulate left E and AC
INC	7020	Increment AC, carry to E
SPA	7010	Skip if AC is positive
SNA	7008	Skip if AC is negative
SZA	7004	Skip if AC is zero
SZE	7002	Skip if E is zero
HLT	7001	Halt computer
INP	F800	Input information and clear flag
OUT	F400	Output information and clear flag
SKI	F200	Skip if input flag is on
SKO	F100	Skip if output flag is on
ION	F080	Turn interrupt on
IOF	F040	Turn interrupt off

m: effective address
M: memory word (operand)
found at m

Machine Language

- A list of instructions or statements for directing the computer to required data processing task.
- Various types of programming:
 - Machine Language
 - ↳ Binary Code
 - ↳ Octal or hexadecimal code
 - Assembly language (Assembler)
 - ↳ Symbolic code
 - High-level language (Compiler)

Comparison Of Programming Language

• Binary Program to Add Two Numbers

Location	Instruction Code
0	0010 0000 0000 0100
1	0001 0000 0000 0101
10	0011 0000 0000 0110
11	0111 0000 0000 0001
100	0000 0000 0101 0011
101	1111 1111 1110 1001
110	0000 0000 0000 0000

• Hex program

Location	Instruction
000	2004
001	1005
002	3006
003	7001
004	0053
005	FFE9
006	0000

• Program with Symbolic OP-Code

Location	Instruction	Comments
000	LDA 004	Load 1st operand into AC
001	ADD 005	Add 2nd operand to AC
002	STA 006	Store sum in location 006
003	HLT	Halt computer
004	0053	1st operand
005	FFE9	2nd operand (negative)
006	0000	Store sum here

• Assembly-Language Program

ORG	0	/Origin of program is location 0
LDA	A	/Load operand from location A
ADD	B	/Add operand from location B
STA	C	/Store sum in location C
HLT		/Halt computer
A, DEC	83	/Decimal operand
B, DEC	-23	/Decimal operand
C, DEC	0	/Sum stored in location C
END		/End of symbolic program

• Fortran Program

```

INTEGER A, B, C
DATA A,83 B,-23
C = A + B
END

```

Assembly Language

↳ Each line has three fields

Label, Instruction Comment
 (max 3 chars) ↳ MVI: //

ADD OPERAND
 ADD Pointer I

↳ Non-MVI:
 register-ref

I/O
 ↳ pseudo-instructions

Pseudo-instruction

↳ Don't execute anything

↳ Just tell computer to do something

ORG N

Hexadecimal number N is the memory location
for the instruction or operand listed in the following line

END

Denotes the end of symbolic program

DEC N

Signed decimal number N to be converted to the binary

HEX N

Hexadecimal number N to be converted to the binary

Example,
Sub two numbers

	ORG 100	/Origin of program is location 100
	LDA SUB	/Load subtrahend to AC
	CMA	/Complement AC
	INC	/Increment AC
	ADD MIN	/Add minuend to AC
	STA DIF	/Store difference
	HLT	/Halt computer
MIN,	DEC 83	/Minuend
SUB,	DEC -23	/Subtrahend
DIF,	HEX 0	/Difference stored here
	END	/End of symbolic program

Translation to Binary Code

Hexadecimal Code		Symbolic Program
Location	Content	
100		ORG 100
101		LDA SUB
102		CMA
103		INC
104		ADD MIN
105		STA DIF
106		HLT
107		MIN, DEC 83
108		SUB, DEC -23
		DIF, HEX 0
		END

Hexadecimal Code		Symbolic Program	
Location	Content		
100	2107	ORG 100	
101	7200	LDA SUB	
102	7020	CMA	
103	1106	INC	
104	3108	ADD MIN	
105	7001	STA DIF	
106	0053	HLT	
107	FFE9	MIN,	DEC 83
108	0000	SUB,	DEC -23
		DIF,	HEX 0
			END

Representation Of Symbolic Program in Memory

↳ Before conversion into binary, assembly program should be saved in the memory.

↳ Loader program is used to input the assembly characters into memory.

↳ Assembly program is in Alphanumeric (ASCII) form in memory

Character	Code	Character	Code	Character	Code
A	41	Q	51	6	36
B	42	R	52	7	37
C	43	S	53	8	38
D	44	T	54	9	39
E	45	U	55	space	20
F	46	V	56	(	28
G	47	W	57	)	29
H	48	X	58	*	2A
I	49	Y	59	+	2B
J	4A	Z	5A	,	2C
K	4B	0	30	-	2D
L	4C	1	31	.	2E
M	4D	2	32	/	2F
N	4E	3	33	=	3D
O	4F	4	34	CR	0D
P	50	5	35		

(carriage return) → End of line code

PL3, LDA SUB I

TABLE 6-11 Computer Representation of the Line of Code: PL3, LDA SUB I

Memory word	Symbol	Hexadecimal code	Binary representation
1	P L	50 4C	0101 0000 0100 1100
2	3 ,	33 2C	0011 0011 0010 1100
3	L D	4C 44	0100 1100 0100 0100
4	A	41 20	0100 0001 0010 0000
5	S U	53 55	0101 0011 0101 0101
6	B	42 20	0100 0010 0010 0000
7	I CR	49 0D	0100 1001 0000 1101

Source Program → Symbolic Assembly Language Program

Assembler

Object Program → Binary Machine Language Program

↳ A two pass assembler scans the entire symbolic program twice.

1. First pass

↳ Generates a table that correlates all user-defined address symbols with their binary equivalent value

↳ To keep track of the location of instructions, the assembler uses a memory word called location counter.

Memory word	Symbol or (LC)*	Hexadecimal code	Binary representation
1	M I	4D 49	0100 1101 0100 1001
2	N ,	4E 2C	0100 1110 0010 1100
3	(LC)	01 06	0000 0001 0000 0110
4	S U	53 55	0101 0011 0101 0101
5	B ,	42 2C	0100 0010 0010 1100
6	(LC)	01 07	0000 0001 0000 0111
7	D I	44 49	0100 0100 0100 1001
8	F ,	46 2C	0100 0110 0010 1100
9	(LC)	01 08	0000 0001 0000 1000

* (LC) designates content of location counter.

↳ Fig 01

Address Symbol	Location
MIN	106
SUB	107
DIF	108

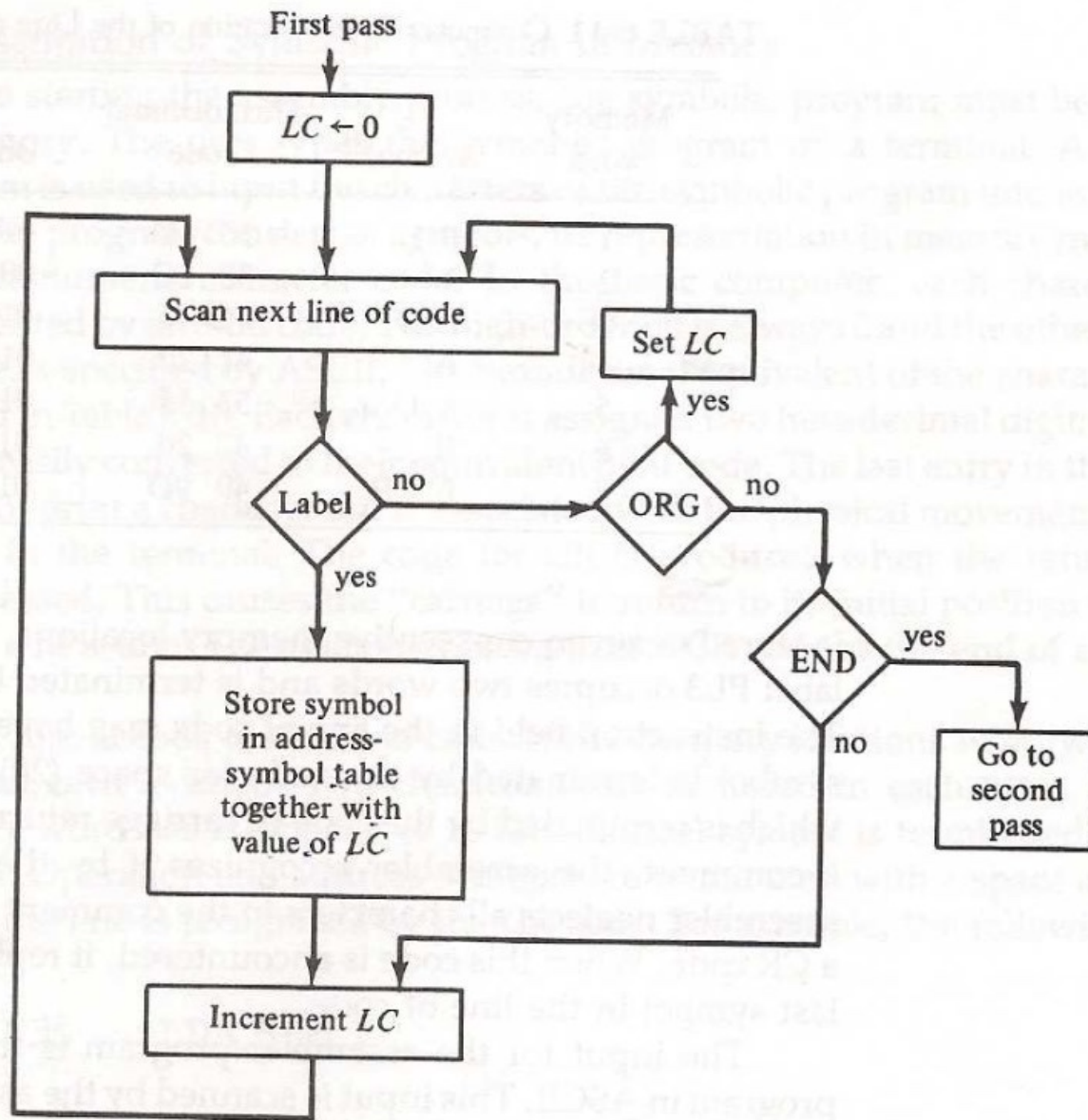
→ According to book

Address Symbol table

→ According to S's

Example for How labels of sub two num program is stored during the first pass

Flow chart for first pass.



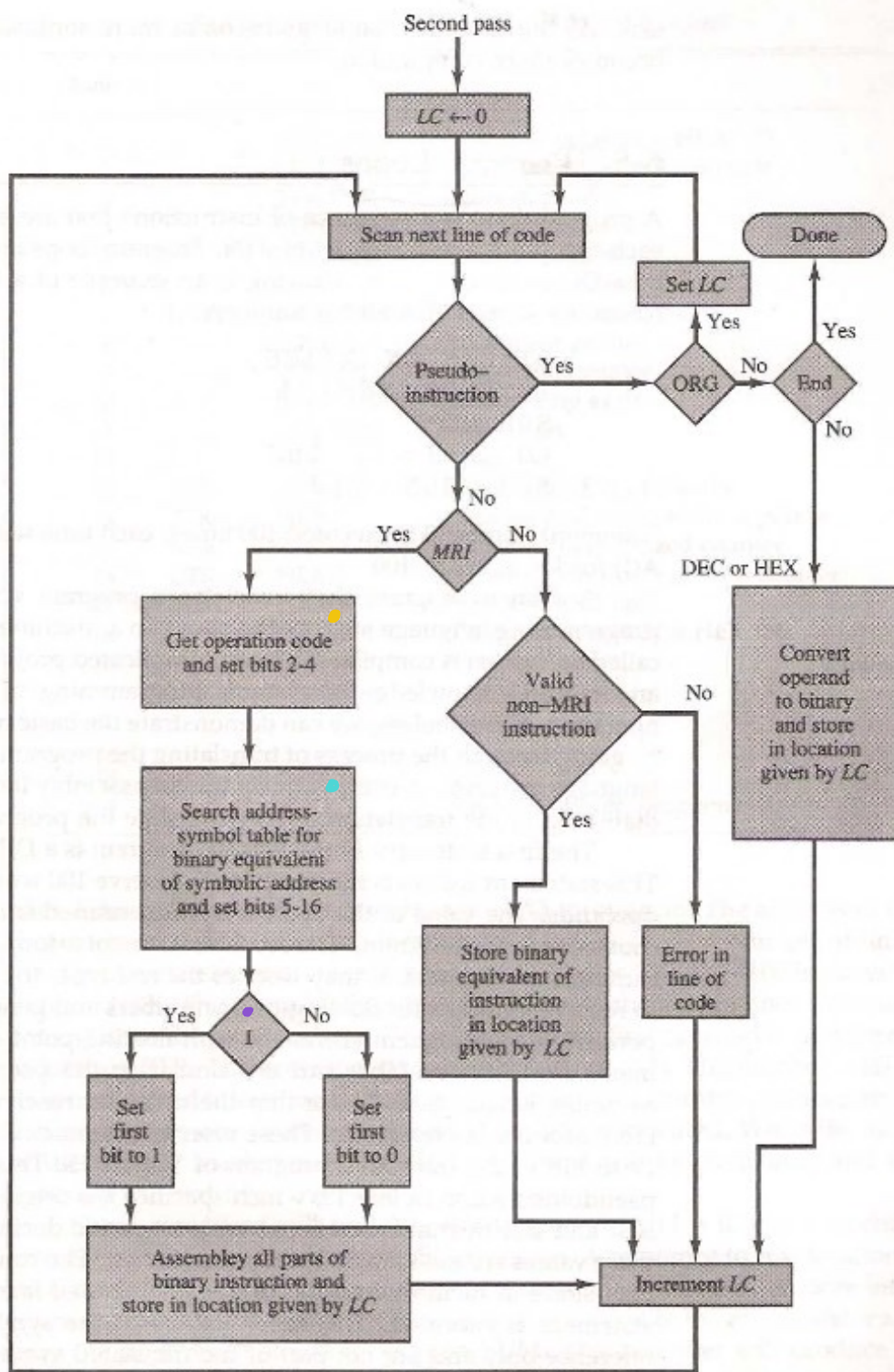
SUB HEX 0

2. Second Pass:

↳ Machine instructions are translated by means of table look-up procedure.

↳ Any symbol encountered in the program must be available in one of these tables.

- Pseudo-instruction table
- MVI table
- Non-MVI table
- Address symbol table.



I opCode Address
1 2 3 4 5 - 16

Program Loops

↳ Sequence of instructions that are executed many times, each with a different set of data.

Line

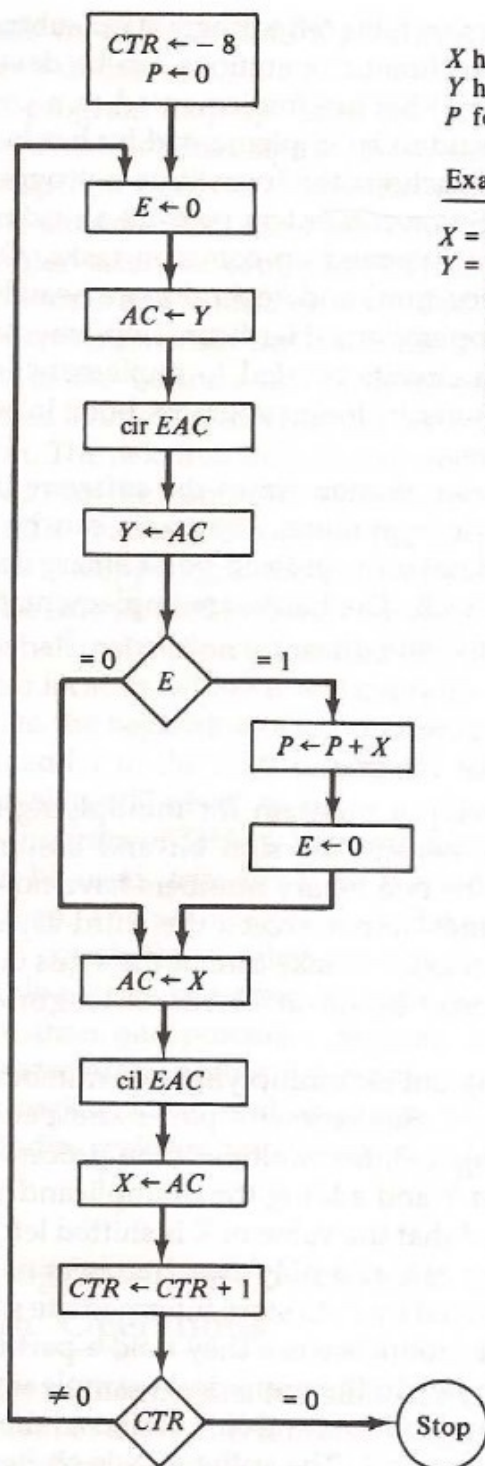
1		ORG 100	/Origin of program is HEX 100
2		LDA ADS	/Load first address of operands
3		STA PTR	/Store in pointer
4		LDA NBR	/Load minus 100
5		STA CTR	/Store in counter
6		CLA	/Clear accumulator
7	LOP,	ADD PTR I	/Add an operand to AC
8		ISZ PTR	/Increment pointer
9		ISZ CTR	/Increment counter
10		BUN LOP	/Repeat loop again
11		STA SUM	/Store sum
12		HLT	/Halt
13	ADS,	HEX 150	/First address of operands
14	PTR,	HEX 0	/This location reserved for a pointer
15	NBR,	DEC -100	/Constant to initialize counter
16	CTR,	HEX 0	/This location reserved for a counter
17	SUM,	HEX 0	/Sum is stored here
18		ORG 150	/Origin of operands is HEX 150
19		DEC 75	/First operand
.			
.			
.			
118		DEC 23	/Last operand
119		END	/End of symbolic program

Programming Arithmetic & Logic Operation

1. Multiplication Program

```

ORG 100
LOP, LDA Y
    CLE
    CTR
    STA Y
    ISZ
    AND ONE
  
```



X holds the multiplicand
 Y holds the multiplier
 P forms the product

Example with four significant digits

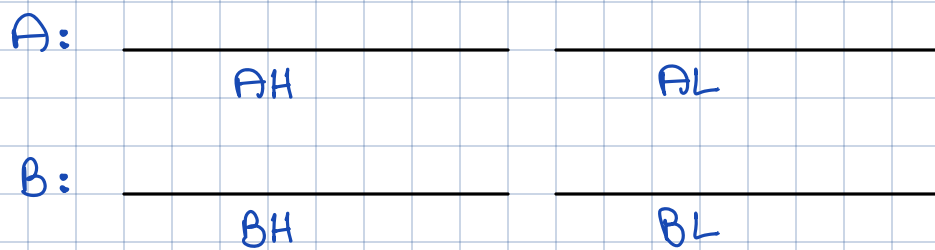
$X = 0000\ 1111$	P
$Y = 0000\ 1011$	$0000\ 0000$
$0000\ 1111$	$0000\ 1111$
$0001\ 1110$	$0010\ 1101$
$0000\ 0000$	$0010\ 1101$
$0111\ 1000$	$1010\ 0101$
$1010\ 0101$	

```

BUN ZRO
ONE, LDA X
    ADD ANS
    STA ANS
    CLE
ZRO, LDA X
    CIL
    STA X
    ISZ CTR
    BUN LOP
    HLT
CTR, DEC -8
X, HEX ---
Y, HEX ---
ANS, HEX ---
END
  
```

	ORG 100	
LOP,	CLE	/Clear E
	LDA Y	/Load multiplier
	CIR	/Transfer multiplier bit to E
	STA Y	/Store shifted multiplier
	SZE	/Check if bit is zero
	BUN ONE	/Bit is one; go to ONE
	BUN ZRO	/Bit is zero; go to ZRO
ONE,	LDA X	/Load multiplicand
	ADD P	/Add to partial product
	STA P	/Store partial product
	CLE	/Clear E
ZRO,	LDA X	/Load multiplicand
	CIL	/Shift left
	STA X	/Store shifted multiplicand
	ISZ CTR	/Increment counter
	BUN LOP	/Counter not zero; repeat loop
	HLT	/Counter is zero; halt
CTR,	DEC -8	/This location serves as a counter
X,	HEX 000F	/Multiplicand stored here
Y,	HEX 000B	/Multiplier stored here
P,	HEX 0	/Product formed here
	END	

2. Double precision Addition



$$\text{Carry, CL} = \text{AL} + \text{BL}$$

$$\text{CH} = \text{AH} + \text{BH} + \text{Carry}$$

```
LOA AL
ADD BL
STA CL
CLA
CIL
ADD AH
ADD BH
STA CH
HLT
```

AL,

BL,

--

CL,

--

AH,

--

BH,

--

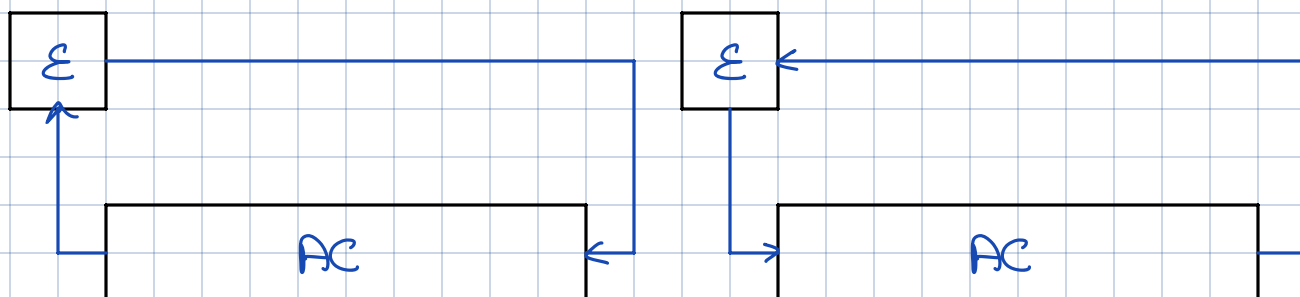
CH,

--

How CIL & CIR work actually?

CIL:

CIR:



Logic Operations

↳ Use AND, CMA, CLA to achieve any logic operation.
for example,

$$\overline{(\overline{x} \overline{y})} = x + y$$

∴ De-Morgan's law

```
LOA A
CMA
STA TMP
LOA B
CMA
STA
AND TMP
CMA
```

Shift Operations:

1. Shift Right

```
CLE
CIL
```

2. Shift Left

```
CLE
CIL
```

3. Arithmetic Right Shift

```
CLE      // set E to 0
SPA      // If not positive
CME      make E 1
CIL      // Move E into AC(15) & move AC to Right
```

4. Arithmetic left shift:

```
CLE
CIL
compare E & AC(15)
if same then good
else throw error (overflow)
```

Subroutines:

↳ A procedure for branching to a sub-routine & returning to main program is called subroutine linkage.

Loc.			
		ORG 100	/ Main program
100		LDA X	/ Load X
101		BSA SH4	/ Branch to subroutine
102		STA X	/ Store shifted number
103		LDA Y	/ Load Y
104		BSA SH4	/ Branch to subroutine again
105		STA Y	/ Store shifted number
106		HLT	
107	X,	HEX 1234	
108	Y,	HEX 4321	
			/ Subroutine to shift left 4 times
109	SH4,	HEX 0	/ Store return address here
10A		CIL	/ Circulate left once
10B		CIL	
10C		CIL	
10D		CIL	/ Circulate left fourth time
10E		AND MSK	/ Set AC(13-16) to zero
10F		BUN SH4 I	/ Return to main program
110	MSK,	HEX FFF0	/ Mask operand
		END	

With Parameters & Data Linkage.

```

ORG 200
LDA X
BSA OR
HEX
STA Y
HLT
X, HEX
Y, HEX 0

OR, HEX 0
CMA
STA TMP
LDA OR I
CMA
AND X
CMA
ISZ OR
BUN OR I

```

Loc.			
		ORG 200	
200		LDA X	/ Load 1st operand into AC
201		BSA OR	/ Branch to subroutine OR
202		HEX 3AF6	/ 2nd operand stored here
203		STA Y	/ Subroutine returns here
204		HLT	
205	X,	HEX 7B95	/ 1st operand stored here
206	Y,	HEX 0	/ Result stored here
207	OR,	HEX 0	/ Subroutine OR
208		CMA	/ Complement 1st operand
209		STA TMP	/ Store in temporary location
20A		LDA OR I	/ Load 2nd operand
20B		CMA	/ Complement 2nd operand
20C		AND TMP	/ AND complemented 1st operand
20D		CMA	/ Complement again to get OR
20E		ISZ OR	/ Increment return address
20F		BUN OR I	/ Return to main program
210	TMP,	HEX 0	/ Temporary storage
		END	

TNP, HEX 0
END

Moving Block of Data

```
ORG
BSA MVE
HEX _____ // Source
HEX _____ // Destination
DEC -16
HLT
```

```
MVE, HEX 0
LOA MVE I
STA PTR1
ISZ MVE
LOA MVE I
STA PTR2
ISZ MVE
LOA MVE I
STA CTR
ISZ MVE
```

} Load values in appropriate variables

```
LOP, LOA PTR1 I
STA PTR2 I
ISZ PTR1
ISZ PTR2
ISZ CTR
BUN LOP
BUN MVE I
```

} Move them

```
PTR1, HEX 0
PTR2, HEX 0
CTR,
```

Input-Output Program

1. Input

```
CIF, SKI
BUN CIF
```

2. Output

```
LOA CTR
COF, SKO
```


INP
 OUT
 STA CHR
 HLT
 CHR, _____

BUN COF
 OUT
 ALT
 CHR, HEX _____

Character Manipulation

↳ Store two characters in one word.

IN2,	--	/ Subroutine entry
FST,	SKI	
	BUN FST	
	INP	/ Input 1st character
	OUT	
	BSA SH4	/ Logical Shift left 4 bits
	BSA SH4	/ 4 more bits
SCD,	SKI	
	BUN SCD	
	INP	/ Input 2nd character
	OUT	
	BUN IN2 I	/ Return

Program to store input chars in buffer

	LDA ADS	/ Load 1 st address of buffer
	STA PTR	/ Initialize pointer
LOP,	BSA IN2	/ Call IN2
	STA PTR I	/ Str double char. W. in buffer
	ISZ PTR	/ Inc Pointer
	BUN LOP	/ BUN to input more char.
	HLT	
ADS,	HEX 500	/ First address of buffer
PTR,	HEX 0	/ Location for pointer

Program to compare two words

LDA WDI	/Load first word
CMA	
INC	/Form 2's complement
ADD WD2	/Add second word
SZA	/Skip if AC is zero
BUN UEQ	/Branch to "unequal" routine
BUN EQL	/Branch to equal routine

WD1, ____

WD2, ____

Program Interrupt

↳ Tasks of interrupt Service Routine

1. Save contents of processor registers
2. Check which flag is set
3. Service the device whose flag is set
4. Restore contents of processor registers
5. Turn interrupt on
6. Return to the running program

Location			
0	ZRO,	—	/Return address stored here
1		BUN SRV	/Branch to service routine
100		CLA	/Portion of running program
101		ION	/Turn on interrupt facility
102		LDA X	
103		ADD Y	/Interrupt occurs here
104		STA Z	/Program returns here after interrupt
.		.	
.		.	
.		.	/Interrupt service routine
200	SRV,	STA SAC	/Store content of AC
		CIR	/Move E into AC(15
		STA SE	/Store content of E
		SKI	/Check input flag
		BUN NXT	/Flag is off, check next flag
		INP	/Flag is on, input character
		OUT	/Print character
		STA PT1 I	/Store it in input buffer
		ISZ PT1	/Increment input pointer
	NXT,	SKO	/Check output flag
		BUN EXT	/Flag is off, exit
		LDA PT2 I	/Load character from output buffer
		OUT	/Output character
		ISZ PT2	/Increment output pointer
	EXT,	LDA SE	/Restore value of AC(15
		CIL	/Shift it to E
		LDA SAC	/Restore content of AC
		ION	/Turn interrupt on
		BUN ZRO I	/Return to running program
	SAC,	—	/AC is stored here
	SE,	—	/E is stored here
	PT1,	—	/Pointer of input buffer
	PT2,	—	/Pointer of output buffer

BOOK Qs.

Q1.

AC	PC	IR
0000	011	7800
C1A5	012	1016
C1A5	014	4014
8184	014	7001
8184	015	1013
8184	013	4013

PC=13
incremented to 14

Q3.

```

ORG 100
LDA A
ADD B
STA SUM
LDA C
CMA
INC
ADD DIF
STA DIF
ADD SUM
STA SUM
HLT
SUM, DEC 0
DIF, DEC 0
A, DEC
B, DEC
C, DEC
END
    
```

Q2.

AC = FFF8

M[103] = 0101

Location	Instruction
100	5103 BSA 103
101	7200 CMA AC =
102	7001 HLT
103	0000 101
104	7800 CLA AC = 0000
105	7020 INC AC = 0001
106	C103 BAN 103 I

1111 1111 1111 1110
F F F 8

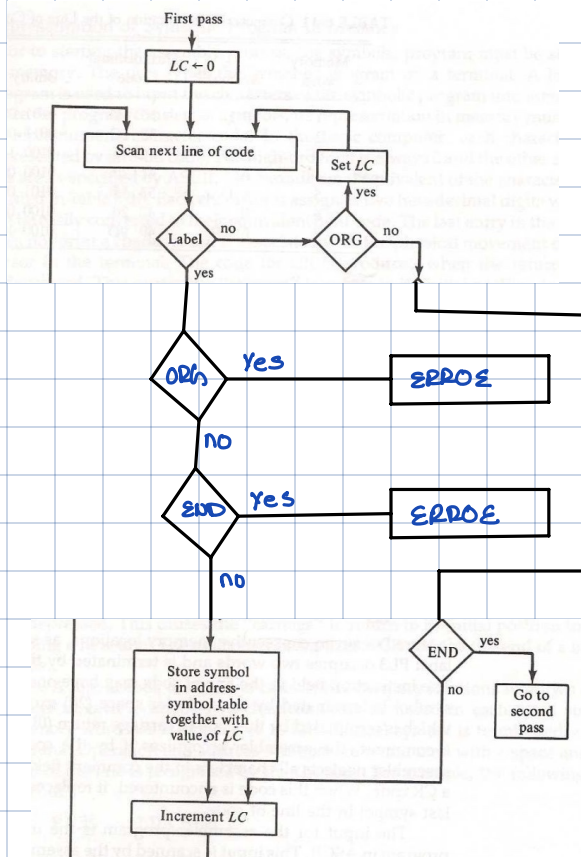
Q4.

Yes, when I appears in the label field it is treated as a symbolic address

In LDA I
I is treated as symbolic address

The assembler will not detect ORG, END if it has a label. Such a label has no meaning.

Q5.



Q6.

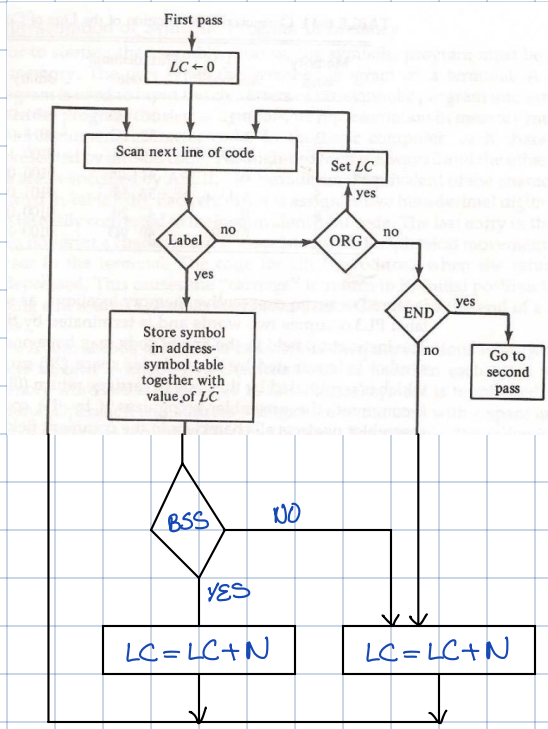
(a) D E
C SPACE
- 3
5 CR

(b) $(-35) = (1111\ 1111\ 1101\ 1101)_2 = (FF0D)$

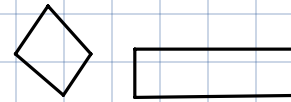
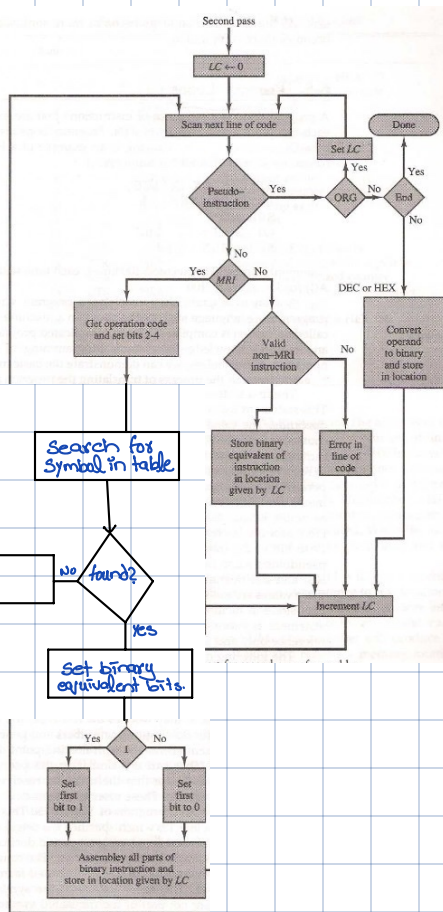
Q7.

LOP	105
ADS	10B
PTR	10C
NBR	10D
CTR	10E
SUM	10F

Q8.



Q9.



Q10.

MRI

A N	41 40
D space	44 20
Value	00 00
A D	41 44
D space	44 20
Value	10 00

Non-MRI

C L	43 4C
A space	41 20
Value	78 00
C L	43 4C
E space	45 20
Value	74 00

Q11.

ORG	100
LOA	B
CMA	
INC	
ADD	A
SPA	
BUN	10
SZA	
BUN	30
BUN	20
HLT	

Q12.
(b)

ROT	107
AGN	10B
STP	10F
CTR	110
WRD	111

(a) Counts numbers of 1s in the WRD

A, DEC
B, DEC
END

Q13.

```

ORh 100
CLF
LOP, STA LOC I
ISZ LOC
ISZ CTR
BUN LOP
HLT
LOC, HEX 500
CTR, DEC -256
END

```

Sol Manual:

```

ORh 100
LDA ADS
STA PTR
LDA NBR
STA CTR
CLF
LOP, STA PTR I
ISZ PTR
ISZ CTR
BUN LOP
HLT
CTR, HEX 0
PTR, HEX 0
ADS, HEX 500
NBR, DEC -256
END

```

Q14.

```

ORh 100
LDA A
SZA
BUN NBR
HLT
NBR, CMA
INC
STA CTR
LOP, LDA P
ADD B
ISZ CTR
BUN LOP
HLT
A, _____
B, _____
CTR, HEX 0
P, HEX 0

```

Sol Manual

```

ORh 100
LDA A
SZA
BUN NBR
HLT
NBR, CMA
INC
STA CTR
CLF
LOP, ADD B
ISZ CTR
BUN LOP
HLT
A, _____
B, _____
CTR, HEX 0
END

```

Q15.

As CTR will be 0 & the program will stop only when CTR will be 0 again which will be when CTR is incremented 65,536 times.

Q16.

	ORh	100
	LOA	NBR
	STA	CTR
	CLA	
LOP,	LDA	Y
	CLE	
	CIR	
	STA	Y
	SZE	
	BUN	ONE
	BUN	ZRD
ONE,	LOA	XL
	ADD	PL
	STA	PL
	CLA	
	CIL	
	ADD	XH
	ADD	PH
	STA	PH
	CLE	
ZRD,	LOA	XL
	CIL	
	STA	XL
	LOA	XH
	STA	XH
	ISZ	CTR
	BUN	LOP
	HLT	
XH,	DEC	
XL,	DEC	
PH,	HEX	0
PL,	HEX	0
Y,	DEC	
NBR,	DEC	-K
CTR,	HEX	0

$E=1,$
 $Count, PL = XL + PL$
 $PH = PH + XH + Count$

$E=0,$
 Left shift XL & XH

Right-shift Y
 & check value
 of E

$C, PL = PL + XL$
 $PH = PH + XH + C$

Left shift XL
 Left shift XH & E from XL in XH

signed Mult

Q17.

	ORh	100
LOP,	LOA	X
	SNA	
	BUN	POS
NEG,	CMA	
	INC	

Q18.

	ORh	100
	LOA	BL
	CMA	
	INC	
	ADD	AL
	STA	C2
	CLA	

$b, CL = AL + BL$ ^{2's comp}
 $CH = AH + \overline{BH} + 1$ ^{1's comp}

```

    STA Y
    LDA X
    CMC
    POS, STA X
    LDA Y
    CLE
    CIR
    SZE
    BUN ONE
    BUN ZRO
    ONE, LDA X
    ADD P
    STA P
    CLE
    ZRO, LDA X
    CIL
    STA X
    ISZ CTR
    BUN LOP
    HLT
    X, DEC
    Y, DEC
    P, HEX 0
    CTR, HEX -7
    END

```

```

    CIL
    STA TMP
    LDA BH
    CMA
    ADD AH
    ADD TMP
    STA CH
    HLT
    TMP, HEX 0
    ;

```

Q19.

$$x \oplus y = x\bar{y} + \bar{x}y = (\overline{x\bar{y}} \cdot \overline{\bar{x}y})$$

$\downarrow$ $\downarrow$
 T1 T2

```

    ORG 100
    LDA Y
    CMA
    AND X
    CMA
    STA T2
    LDA X
    CMA
    AND Y
    CMA
    AND T1
    CMA
    STA A

```

X, _____
 Y, _____

Q20.

Arithmetic left shift

```

    ORG 100
    CLE
    CIL
    SZE
    BUN ONE
    BUN ZRO
    ONE, SNA
    BUN OVF
    EXT, HLT
    ZRO, SPA
    BUN OVF
    BUN EXT

```

A,
T1,

Q21.

```

ORG 100
BSA SUB
HEX       
HEX       
STA Y
HLT
SUB, HEX 0
LOA SUB I
CMA
INC
ISZ SUB
ADD SUB I
ISZ SUB
BUN SUB I
Y, HEX 0
END

```

B → subtrahend
A → minuend
A - B

Q22.

```

ORG 100
BSA COM
HEX        ←
DEC       
HLT
COM, HEX 0
LOA COM I
STA PTR
ISZ COM
LOA COM I
STA CTR
LOP, LOA PTR I
CMA
STA PTR I
ISZ PTR
ISZ CTR
BUN LOP
BUN COM I
PTR, HEX 0
CTR, HEX 0
END

```

Q23.

```

ORG 100
LOA X
BSA ROT
HLT
ROT, HEX 0
CTR
CIR
CIR
CIR
BUN ROT I
A, HEX 079C
END

```

Σ=1	0000	0111	1001	1100
①				
Σ=0	1000	0011	1100	1110
②				
Σ=0	0100	0001	1110	0111
③				
Σ=1	0010	0000	1111	0011
④				
Σ=1	1001	0000	0111	1001
	9	0	7	9

Q24.

```

ORG 100
LOA NBR
CMA
INC
STA CTR
LOA ADS

```

Q25.

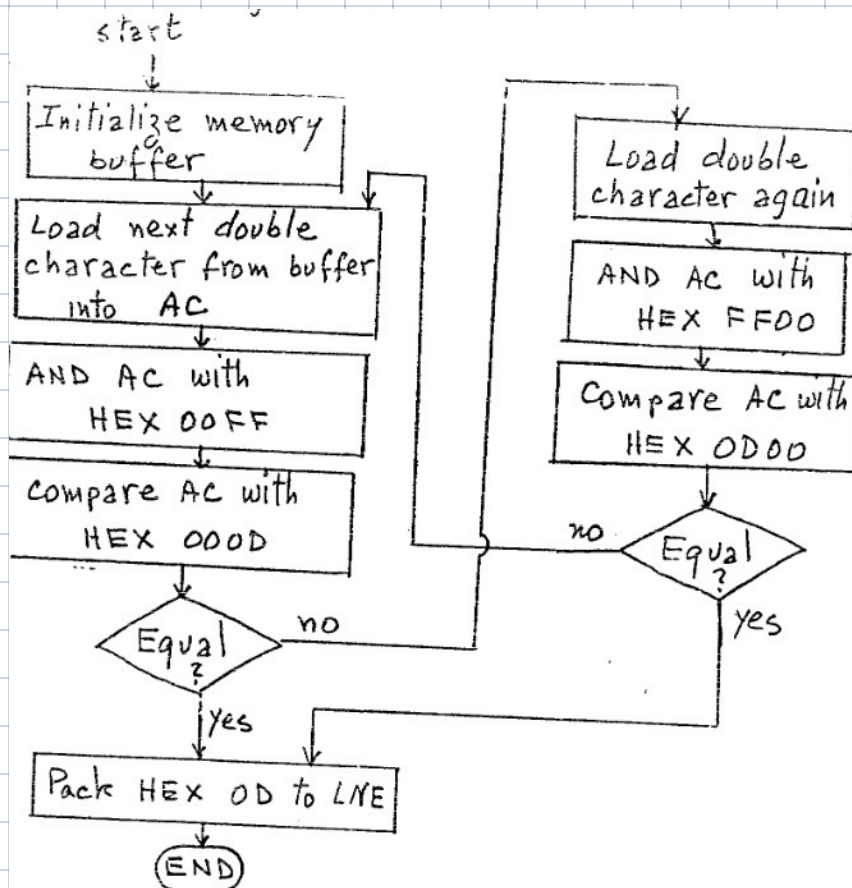
```

ORG 100
LOA WRD
AND MSK1
STA CH2
LOA NRD
AND MSK2

```


	STA	PTR		CIR	
LOP,	BSA	IN2		CIR	
	STA	PTR I		CIR	
	ISZ	PTR		CIR	
	ISZ	CTR		CIR	
	BUN	LOP		CIR	
	HLT			CIR	
IN2,	HEX	0		CIR	
FW,	SKI			STA	CA1
	BUN	FW		HLT	
	INP		MSK1,	HEX	00FF
	OUT		MSK2,	HEX	FF00
	BSA	LSH	WRD,	HEX	_____
	SKI		CH1,	HEX	_____
	BUN	SW	CH2,	HEX	_____
	IUP			END	
	OUT				
	BUN	IN2 I			
ADS,	HEX	400			
NBR,	DEC	512			
CTR,	HEX	0			
PTR,	HEX	0			
	END				

Q 26.



Sol manual:

Q28. 0 HEX 0
 1 BUN SRV
 ORH 100
 LDA A
 ION
 ADD B
 :

SRV, STA SAC
 CIR
 STA SE
 LDA MOD
 SKI
 CMA
 SZA
 BUN OPT
 INP
 STA CH
 OPT, SKO
 BUN EXT
 SZA
 BUN EXT
 LDA CH
 OUT
 EXT, LDA SE
 CIL
 LDA SAC
 BUN O I
 END

0 HEX 0
 1 BUN SRV
 ORH 100
 LDA A
 ION
 ADD B
 :

SRV, STA SAC
 CIR
 STA SE
 LDA MOD
 CMA
 SZA
 BUN NXT
 SKI
 BUN NXT
 INP
 OUT
 STA CH1
 BUN EXT
 NXT, LDA MOD
 SZA
 BUN EXT
 SKO
 BUN EXT
 LDA CH
 OUT
 EXT, LDA SE
 CIL
 LDA SAC
 ION
 BUN O I



Q15.

↳ After program is executed once then value at location CTR will be 0.

↳ When program runs again, it will be zero on overflow which is after being incremented $2^{16} = 65,536$ times.

↳ Fix as follows

```
ORH    0
LOA     NBR
STA     CTR
CLA
STA     P
:
CTR HEX 0
NBR DEC -8
P  HEX 0
:
END
```


4. C1A5 014 7001

5. 8184 015 1017

6. 8184 013 4013

Q3.

ORG = 0

```
LDA SUM
ADD A
ADD B
STA SUM
LDA C
CNC
INC
ADD DIF
STA DIF
LDA SUM
ADD DIF
STA SUM
HLT
```

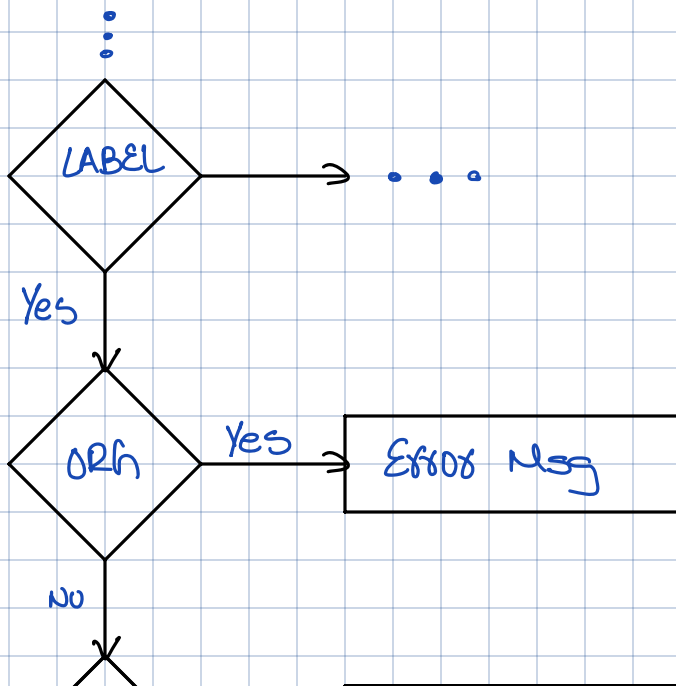
```
SUM, HEX 0
DIF, HEX 0
A, DEC 1
B, DEC 2
C, DEC -3
END
```

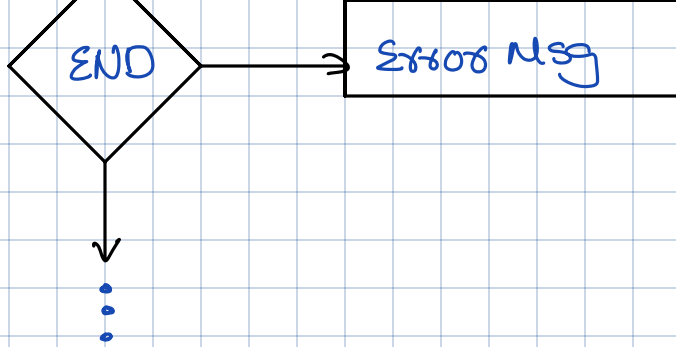
Q4. Yes, if we write I in the symbolic address field
for example,

LDA I Here I is Address

LDA I I Here 2nd I
is indirect bit

Q5.





Q6. DEC, -35

(a)

D E	44 45	0100 0100 0100 0101
C	43 20	0100 0011 0010 0000
- 3	20 33	0010 1101 0011 0011
5	35 00	0011 0101 0000 1101

(b) $(35)_{10} = (0000\ 0000\ 0010\ 0010)_2$
 $-35 = 1111\ 1111\ 1101\ 1101 = (FFD6)_{16}$

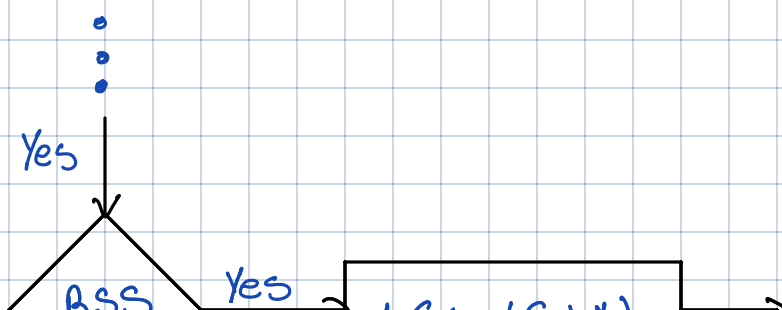
Q7.

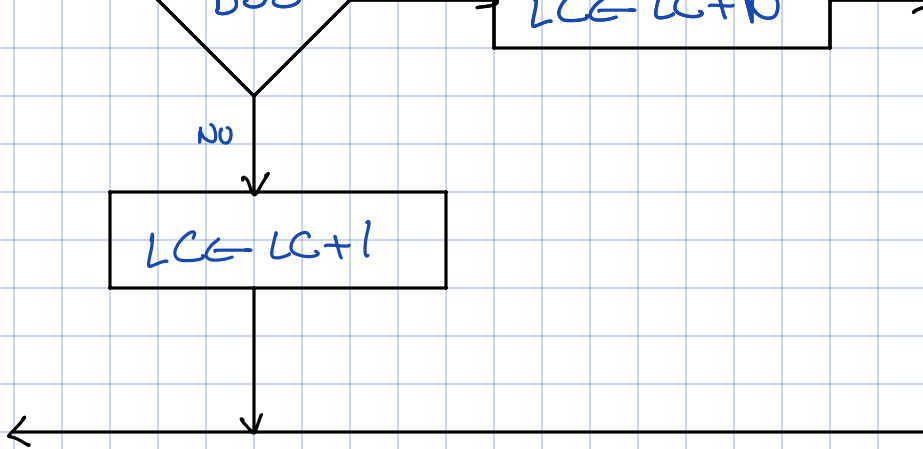
(a)

LOP	105
ADS	10B
PTR	10C
NBR	10D
CTR	10E
SUM	10F

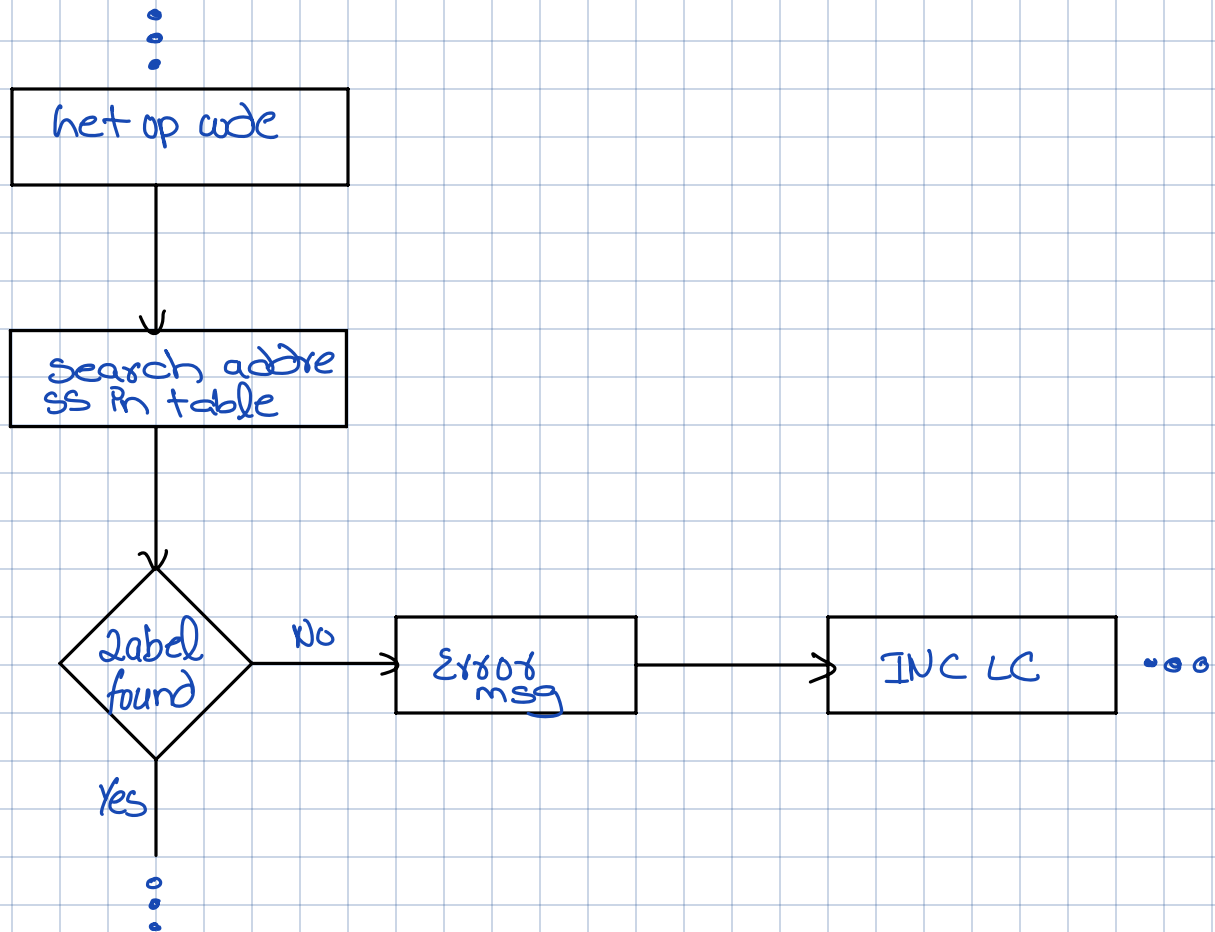
(b) Write till HLT in HEXA

Q8.





Q9.



Q10.

Look Fig 01. Same as that

Q11.

ORG 0	BOOK
LDA B	LDA B
CNC	CNC
INC	INC
ADD A	ADD A
STA C	STA C
	SPA
GIL A	BUN N10

Binary Here!

SZE
 BUN ONE +02 0
 BUN ZERO -
 ZERO, BUN 20
 ONE, LDA C
 SZA
 BUN 30
 BUN 20
 ALT

SZA
 BUN N30
 BUN N20

Q12.

(9)

	ORG 100		
	CLE	clear E	E = 0
	CLA	clear AC	AC = 0
	STA CTR	store AC in CTR	CTR = 0
	LDA WRD	load WRD in AC	WRD = 62C1 ← AC
	SZA X	skip if AC is 0	
	BUN ROT	move to ROT	
	BUN STP	move to STP	
ROT,	CIL	circular left shift AC	AC = C582
	SZE ✓	skip if E is 0	AC = 8A05
	BUN AGN X	move to AGN	
	BUN ROT	move to ROT	
AGN,	CLE	clear E	E = 0
	ISZ CTR	skip if CTR is 0	CTR = 1
	SZA	skip if AC is 0	
	BUN ROT	move to ROT	
STP,	HLT	stop	
CTR,	HEX 0		
WRD,	HEX 62C1		
	END		

0110 0010 1100 0001

1100 0101 1000 0010 , $\epsilon = 0$
 5 8 2

²1000 ^{3 4 5}1011 ⁶0000 0101 , $\epsilon = 1$
 8 A 0 5

Q13.

```

ORH 100
LOA NBR
STA CTR
LOA ADS
STA PTR
LOP, CLA
     STA PTR
     ISZ PTR
     ISZ CTR
     BUN LOP
     HLT
ADS  HEX 300
PTR  HEX 0
CTR  HEX 0
NBR  DEC -256
END

```

Q14.

```

ORH 0
LOA B
SZA
BUN NZR
HLT
NZR, CMC
     INC
     STA CTR
     CLA
LOP, ADD A
     ISZ CTR
     BUN LOP
     STP PRO
     HLT
A, DEC 5
B, DEC 4
CTR, HEX 0
PRO, HEX 0

```